

Karim Baba.

San Francisco, CA

✉ founder@karimbabasf.com

in [LinkedIn](#)

✂ X / [@karimbabasf](#)

[GitHub](#)

📩 [Telegram](#)

TO THE STRATEGIC PROJECTS TEAM, TAILSCALE

18 AUGUST 2026

Hello,

Your posting asks candidates to say where they are strongest and where they want to develop, so I will do exactly that. I am strongest at the half of the work most people find tedious: taking something that already demos well and closing the long tail behind it. I direct coding agents all day, which means I spend most of my time verifying rather than typing, and I have opinions about that earned from being wrong in production.

■ WHERE I AM STRONGEST

- › **Prototype to production is the part I stay for.** I built a voice agent that answers the phone for a business I co-run. It handled a real call within a day, and the two weeks after that were the whole job: an outbox table because the messaging API caps a group at roughly 20 messages a minute shared across every thread, so direct sends pass in testing and fail in a burst; a detect-and-recreate path because any admin can permanently delete a thread with no recovery; dual-channel recording, chosen at the start because it cannot be fixed later; 476 tests and a 22-check preflight before deploy.
- › **I have already shipped a gateway that decides what an agent may do.** Phosphor, my own MCP server, exposes 38 tools behind a permission gate, runs on mainnet against real balances, and carries 993 tests. What leaves the process is built by an allowlist projection rather than scrubbed on the way out, so a field added next quarter is invisible by default instead of shipping until somebody remembers it. Reading Aperture's description was the first time a job posting described the thing already on my disk.
- › **I know where a green test suite lies.** A control that asks the backend before it renders draws nothing in a browser harness, so 392 passing tests once proved nothing about the change I had just made. Making that state reachable in the harness exposed two real defects in a single look. In the same spirit, I accepted a 13 times faster indexing path only after replaying old and new code against a copy of the real store and diffing all 4,019 rows for zero differences.
- › **I ship in public and I read other people's code closely.** Warden and my agent skill library are public MIT repositories. I also found and fixed a secret-exfiltration flaw in an open-source MCP server, where the guard matched tracked secrets as a literal substring, so any encoding defeated it while still round-tripping to the exact secret. That work is with the maintainer under coordinated disclosure.

■ HOW I WORK WITH AGENTS

Agents made the cheap half of engineering free and moved the entire cost to verification, so the review practice is the product now. In my own repositories that means invariants enforced by a test over the generated output rather than by a comment somebody can ignore, canary tests over the exact strings that must never leave the process, harnesses that score the real prompt against the real model instead of a paraphrase of it, and a standing rule that a passing suite is not evidence when the new state is unreachable inside the harness. I also verify what is actually running before I read a line of code, because a feature reported broken the morning after I shipped it turned out to be a two-week-old binary.

■ WHERE I WANT TO DEVELOP, AND WHY HERE

Straight answer on the must-have: I have not shipped production Go. I write Rust, TypeScript and Python daily, and I picked up Rust because Warden needed it, then shipped that app publicly five days after the first commit. Go is the next one, and I would rather tell you that now than have it surface in week two. Networking depth is the other place I want a stronger team around me.

I want this role because MCP gateways, LLM proxies and sandboxes are what I already build on my own time, and because I would rather be held to somebody else's production bar than to my own. If a different team, level or scope inside Tailscale is the better place to start that, I would welcome that conversation just as much.

THANK YOU FOR YOUR TIME,

Karim Baba.